# Profiling from Zero

A Complete Hands-On Guide to perf, Nsight, gprof, Valgrind, VTune, and beyond

Prepared for CK | IIT Delhi | Nanopore Project
Prof. Kolin Paul | Summer 2025

Batch 4 · VTune, System Tools, and Synthesis · Chapters 16 to 20

## Contents



*This batch assumes Chapters 1 to 15: the full perf toolkit, the Valgrind suite, gprof, and all Nsight tools (Systems and Compute), plus the five benchmark programs.*

CHAPTER 16

# Intel VTune Profiler

We open the final batch with one more CPU profiler, and you might reasonably ask why, after perf and the entire Valgrind suite, we need another. The answer is one idea so valuable it justifies the whole chapter: the Top-Down Methodology. perf gives you raw counters and leaves you to interpret them. VTune takes those same counters and organizes them into a decision tree that walks you from "the program is slow" to "the specific microarchitectural reason it is slow" without requiring you to memorize which counter means what. It is the difference between being handed a pile of medical test results and being handed a diagnosis. VTune also brings a polished GUI, richer source-and-assembly views, and analysis types tuned to specific questions, but the Top-Down Methodology is the reason it earns a place in your toolkit even after you know perf cold.

A note on honesty before we start. VTune is an Intel tool, tuned for Intel CPUs, and it reads Intel-specific counters that encode the Top-Down model in hardware. On the lab Dell Precision with its Intel Xeon, VTune works fully. On an AMD machine it works in reduced form, and on the GPU it does not apply at all. So VTune is a CPU-side tool, most powerful on Intel hardware, and for our project that makes it a Kraken-2 tool, the same niche as perf and Cachegrind, but with the Top-Down lens that the others lack.

## 16.1 What VTune Adds Over perf

Everything VTune measures, perf can also measure, because they both read the same hardware counters. What VTune adds is interpretation, presentation, and a few conveniences.

- **The Top-Down Methodology (TMA):** a hierarchical breakdown of where every CPU cycle went, organized so you drill from coarse categories to specific causes. This is the headline feature and the rest of the chapter.
- **A GUI-first workflow:** sortable function lists, clickable source-and-assembly views, timeline visualizations, and the ability to click a hotspot and immediately see the annotated assembly with per-instruction counts. perf can do most of this in its TUI, but VTune's presentation is far more legible for complex programs.
- **Named analysis types:** instead of remembering which events to pass to perf stat, you pick an analysis like Hotspots, Memory Access, or Microarchitecture Exploration, and VTune selects and interprets the right counters for that question.
- **Precise memory attribution with PEBS:** like perf mem, VTune uses precise event sampling to attribute memory latency to specific loads and data structures, but with a richer interface for seeing which arrays cause the misses.

Installation comes through the Intel oneAPI toolkit, which bundles VTune with Intel's compilers and libraries. After installing, you source the environment script to put the tools on your path.

```bash
# after installing the Intel oneAPI Base Toolkit
source /opt/intel/oneapi/vtune/latest/env/vars.sh

vtune --version

# VTune also needs the sampling driver or perf access; on Linux it can
# use the same perf_event interface, so the perf_event_paranoid setting
# from Chapter 3 applies here too.
```

## 16.2 Analysis Types

VTune frames profiling as choosing an analysis type, each of which answers a specific question by collecting and interpreting the right counters. The four that matter most map cleanly onto the four questions from Chapter 1.

- **Hotspots:** where is time spent? A sampling analysis that ranks functions by time, the VTune equivalent of perf record. This is the where question and the usual starting point.
- **Microarchitecture Exploration:** why is time spent there? This runs the Top-Down Methodology and is the why question. It is VTune's signature analysis and the heart of this chapter.
- **Memory Access:** a memory-focused analysis that attributes traffic and latency to specific data structures using PEBS, telling you which arrays cause the cache misses. This is the deep version of the why question when the answer is memory.
- **Threading:** a concurrency analysis showing the thread timeline, lock contention, and idle time, for diagnosing why a parallel program does not scale. This is the question that matters for Kraken-2 at high thread counts and for our OpenMP matmul.

You run an analysis on the command line, naming the type and a result directory, then generate a report from the result. The GUI consumes the same result directory, so you can collect on a headless server and view on your laptop.

```bash
gcc -O2 -g -o matmul_naive matmul_naive.c

# Hotspots: where is the time?
vtune -collect hotspots -result-dir r_hotspots ./matmul_naive
vtune -report hotspots -result-dir r_hotspots

# Microarchitecture Exploration: why? (the Top-Down analysis)
vtune -collect uarch-exploration -result-dir r_uarch ./matmul_naive
vtune -report summary -result-dir r_uarch

# Memory Access: which data structures cause the misses?
vtune -collect memory-access -result-dir r_mem ./matmul_naive
```

## 16.3 The Top-Down Methodology

Now the centerpiece. The Top-Down Methodology, often abbreviated TMA, is a way of accounting for every cycle the CPU spent, organized as a tree. At the top, every cycle is classified into one of four categories. Each category that turns out to be significant is then broken down into subcategories, and those into finer causes, so you descend the tree following the largest number at each level until you reach a specific, actionable cause. It is a guided diagnosis that converts the cycle into the unit of accounting and asks, at each level, where did the cycles go.

## Level 1: the four fates of every cycle

At the top level, the CPU's issue slots, the opportunities to execute a micro-operation each cycle, are classified into four buckets. A modern core has several issue slots per cycle, and each one, every cycle, is either doing useful work or being wasted in one of three ways.

- **Retiring:** the slot did useful work that contributed to the program's progress. This is the good bucket. A high retiring percentage means the CPU is genuinely getting work done. If your program is mostly retiring, it is efficient and the only way to speed it up is to do less work or use better instructions.
- **Bad Speculation:** the slot did work that was thrown away because of a branch misprediction or a pipeline clear. The CPU speculated down the wrong path. High bad speculation points at unpredictable branches, the territory of the branch-miss rate from Chapter 3.
- **Front End Bound:** the slot was idle because the front end could not deliver instructions fast enough, due to instruction-cache misses, decoding bottlenecks, or branch-target issues. The back end was ready but starved of work to do.
- **Back End Bound:** the slot was idle because the back end could not accept more work, almost always because it was waiting on data from the memory system or because the execution units were saturated. This is where memory-bound programs land, and it is the most common bucket for the kind of numerical code we profile.

The beauty of this top level is that the four buckets are exhaustive and mutually exclusive: every cycle's issue slots fall into exactly one, and they sum to 100 percent. So the first glance at a TMA report tells you, with no interpretation needed, what fraction of your CPU's potential was wasted and in which of the three ways. A program that is 80 percent Back End Bound has a memory or execution-unit problem; a program that is 30 percent Bad Speculation has a branch problem; a program that is 90 percent Retiring is already efficient and you should stop optimizing the microarchitecture and start optimizing the algorithm.

## Level 2 and below: drilling into Back End Bound

When Level 1 says Back End Bound, which is the common case for numerical code, you descend. Level 2 splits Back End Bound into two children.

- **Memory Bound:** the back end stalled waiting for the memory system. This is the cache-miss story from Chapters 2 and 3, now precisely quantified as a fraction of total cycles.
- **Core Bound:** the back end stalled because the execution units themselves were saturated or because of data dependencies, not because of memory. This is the closest thing to

compute-bound in the TMA model.

When Level 2 says Memory Bound, you descend again to Level 3, which splits the memory stalls by which level of the hierarchy caused them: L1 Bound, L2 Bound, L3 Bound, DRAM Bandwidth Bound, DRAM Latency Bound, and Store Bound. Now you are at the actionable bottom of the tree. "DRAM Bandwidth Bound" tells you the memory bus is saturated and the fix is to reduce traffic. "L3 Bound" tells you the working set overflows L2 but fits in L3 and the fix is better blocking. "DRAM Latency Bound" tells you you are waiting on individual scattered accesses, the random-access pattern, and the fix is to improve locality or prefetching. Each leaf of the tree maps to a specific optimization, which is exactly what makes TMA so valuable: it does not just tell you that you are slow, it walks you to the precise reason and therefore the precise fix.

> **Key Insight**
> The Top-Down Methodology turns the vague counters of perf into a guided descent. Start at the top: which of the four buckets dominates? Follow the largest number down the tree, level by level, until you reach a leaf like DRAM Bandwidth Bound or L3 Bound. The leaf names the fix. You never have to remember which raw counter means what, because the tree does the interpretation. This is why VTune is worth knowing even after perf: it encodes the expert's interpretation of the counters into a structure a beginner can follow.

## 16.4 Walking Our Benchmarks Through TMA

Let me run both matmul versions through the Top-Down tree, because the contrast is the clearest possible illustration of what TMA reveals, and it confirms everything we found with perf and Cachegrind from a new angle.

### matmul_naive through TMA

Level 1: the report shows a small Retiring percentage, perhaps 15 percent, and a dominant Back End Bound, perhaps 80 percent. The CPU is wasting most of its issue slots in the back end. Descend. Level 2: Back End Bound splits into Memory Bound at, say, 72 percent and Core Bound at 8 percent. The stall is overwhelmingly memory, not execution. Descend. Level 3: Memory Bound splits, and DRAM Bandwidth Bound dominates, because the naive kernel re-reads B from memory with no reuse and saturates the memory bus. The leaf is DRAM Bandwidth Bound. The diagnosis, reached by following the largest number down three levels, is that naive matmul wastes most of its cycles waiting on saturated DRAM bandwidth caused by the stride-n access to B. This is the identical conclusion we reached with perf's low IPC and high cache-miss rate and with Cachegrind's exact miss counts, now expressed as a clean path down the TMA tree to a named leaf.

### matmul_ikj through TMA

Level 1: the report now shows a high Retiring percentage, perhaps 70 percent or more, and a much smaller Back End Bound. The CPU is mostly doing useful work. There is no need to descend far, because the top level already tells the story: the program is efficient, retiring most of its issue slots,

with only modest remaining stalls. The TMA tree for ikj is shallow because there is no deep problem to find. That shallowness is itself the result: when Level 1 is dominated by Retiring, you are near the microarchitectural ceiling and further speedup must come from the algorithm, not from fixing stalls. Compare the two trees side by side and the optimization is visible as the collapse of the Back End Bound branch and the growth of the Retiring branch, the cycles migrating from wasted-on-memory to spent-on-work.

> **How This Connects to Your Project**
>
> For the Kraken-2 report page, running VTune's Microarchitecture Exploration and capturing the Top-Down breakdown gives you a uniquely clear statement of the bottleneck. The hash-lookup workload, with its random access into a multi-gigabyte table, will almost certainly descend to a memory leaf, but which one is informative. If it lands on DRAM Latency Bound, that confirms the problem is scattered individual accesses each waiting hundreds of cycles, the random-probe pattern, which is exactly what the Hot-K-mer cache addresses by making the hot accesses local. If it lands on DRAM Bandwidth Bound, the bus itself is saturated.
>
> The report sentence becomes precise: "VTune Top-Down analysis classifies Kraken-2 as X percent Back End Bound, of which the dominant leaf is DRAM Latency Bound, confirming the bottleneck is scattered random accesses into the hash table rather than bus saturation." That distinction between latency-bound and bandwidth-bound, which TMA makes cleanly and which raw perf counters leave you to infer, directly shapes whether the recommended fix is improving locality (caching) or reducing total traffic. On the Intel Xeon of the lab Dell Precision, VTune runs at full capability and this analysis is straightforward to produce.

## 16.5 The Threading and Memory Access Analyses

Two more analyses deserve mention because they answer questions the others cannot. The Threading analysis shows, for a parallel program, a timeline of each thread with the regions where threads were running, waiting on locks, or idle. It computes the effective parallelism and highlights serialization, lock contention, and load imbalance. For Kraken-2 at thirty-two threads, or for our OpenMP matmul, this is how you diagnose why adding threads stops helping: the Threading analysis shows whether the threads are genuinely working in parallel or spending their time waiting on a shared lock or sitting idle because the work was unevenly divided. It is the VTune complement to perf c2c from Chapter 5, focused on the lock-and-idle side rather than the cache-line side of parallel inefficiency.

The Memory Access analysis attributes memory latency not just to functions but to *data structures*. Using precise sampling, it can tell you that the misses come from accesses to a specific array, named by its allocation site. For Kraken-2 this could pinpoint the hash table array as the source of the misses, which is unsurprising but confirms the target, and more usefully it can reveal whether secondary data structures contribute unexpected traffic. Attributing misses to a named data structure rather than just a function is a level of detail that perf and Cachegrind do not easily provide, and it is occasionally the clue that reveals an unexpected source of memory traffic hiding behind the obvious one.

### Common Pitfall

Expecting VTune to tell you something perf could not. They read the same counters; VTune's advantage is interpretation via TMA, not access to secret data. If you already know how to read perf counters expertly, VTune mainly saves you interpretation effort and presents it more clearly.

Running VTune on an AMD CPU or a non-Intel system and expecting full TMA. The Top-Down model relies on Intel-specific counters; on other hardware VTune works in reduced form. On the lab's Intel Xeon it is full-featured.

Reading only the Level 1 TMA summary and stopping. The value is in descending the tree to a leaf. A program that is 80 percent Back End Bound tells you little; descending to DRAM Latency Bound versus DRAM Bandwidth Bound tells you the fix.

Forgetting -g, which leaves VTune unable to map hotspots to source lines, the same mistake as with perf and Cachegrind. Always compile profilable builds with -O2 -g.

### What We Just Learned

VTune reads the same hardware counters as perf but adds interpretation, a GUI, named analysis types, and above all the Top-Down Methodology. On the lab's Intel Xeon it is full-featured; it is a CPU-side, Intel-focused tool, making it a Kraken-2 tool for our project.

Analysis types map to the four questions: Hotspots for where, Microarchitecture Exploration for why (this runs TMA), Memory Access for which data structures, and Threading for parallel scaling.

The Top-Down Methodology classifies every CPU issue slot into four exhaustive buckets: Retiring (useful work), Bad Speculation (mispredicted branches), Front End Bound (starved of instructions), and Back End Bound (stalled on memory or execution). They sum to 100 percent.

When Back End Bound dominates, descend: Level 2 splits into Memory Bound versus Core Bound; Level 3 splits Memory Bound into L1, L2, L3, DRAM Bandwidth, DRAM Latency, and Store Bound. Each leaf names a specific fix.

matmul_naive descends to DRAM Bandwidth Bound, confirming the stride-n B access saturates memory. matmul_ikj is dominated by Retiring with a shallow tree, meaning it is near the microarchitectural ceiling. The optimization is visible as cycles migrating from Back End Bound to Retiring.

For Kraken-2, TMA distinguishes DRAM Latency Bound (scattered random accesses, fixed by caching for locality) from DRAM Bandwidth Bound (bus saturation), a distinction that directly shapes the recommended fix and that raw perf counters leave you to infer.

### Before You Move On

You now have a third lens on CPU performance, and the most beginner-friendly one for the why question, because TMA guides the diagnosis rather than handing you raw counters. Between perf, Valgrind, and VTune, you can attack any CPU bottleneck from three complementary angles.

The next chapter zooms out from dedicated profilers to the humble Linux system tools that frame every investigation: time, strace, ltrace, vmstat, numactl, and the contents of /proc. These are the tools you run first, before calling in the heavy profilers, to answer the basic triage questions: is this CPU-bound or memory-bound, is it swapping, how many system calls is it making, is it suffering on a NUMA machine. They are unglamorous and indispensable, the stethoscope before the MRI.

CHAPTER 17

# time, strace, and Linux System Tools

Before you call in a heavy profiler, there is a layer of simple Linux tools that answer the basic triage questions in seconds, with no setup and no overhead. These are the stethoscope before the MRI. They will not tell you which cache line is hot, but they will tell you whether your program is CPU-bound or memory-bound, whether it is swapping to disk, how many system calls it is drowning in, whether it is suffering on a NUMA machine. Most performance investigations should start here, because these tools are fast to run and they often reveal that the problem is something a profiler would have missed entirely: the program is paging, or it is making a million tiny system calls, or it is bound by disk I/O that no CPU profiler even sees. This chapter is the unglamorous, indispensable first layer of any investigation.

Name the wrong instinct, because it is the inverse of the usual one. People who have just learned perf and Nsight reach for them immediately, for everything, including problems those tools cannot see. If your program is slow because it is swapping to disk, perf stat will show you a low IPC and you will hunt for cache misses that are not the real problem, when a five-second run of vmstat would have shown the swap activity instantly. The heavy tools see inside the CPU; the system tools see the program's relationship with the operating system and the hardware around it. You need both, and you should look at the cheap, wide system view before the expensive, narrow CPU view.

## 17.1 /usr/bin/time: The First Command You Run

Everyone knows the shell builtin *time*, which prints three lines of wall, user, and system time. Almost nobody uses its far more informative cousin, */usr/bin/time -v*, the actual binary rather than the builtin, with the verbose flag. It reports not just timing but memory usage, page faults, context switches, and I/O, a complete one-line triage of how the program behaved.

bash
```
/usr/bin/time -v ./matmul_naive 2>&1 | tail -20
```

output
```
    Command being timed: "./matmul_naive"
    User time (seconds): 7.81
    System time (seconds): 0.01
    Percent of CPU this job got: 99%
    Elapsed (wall clock) time (h:mm:ss or m:ss): 0:07.84
    Maximum resident set size (kbytes): 26512
    Major (requiring I/O) page faults: 0
    Minor (reclaiming a frame) page faults: 6189
    Voluntary context switches: 1
    Involuntary context switches: 38
    File system inputs: 0
    File system outputs: 0
```

Read this as a triage. *User time 7.81, System time 0.01*: the program spends essentially all its time in its own code, not in the kernel, so it is a compute or memory problem, not a system-call problem. *Percent of CPU 99 percent*: it kept one core busy the whole time, so it is not waiting on I/O or locks. *Maximum resident set 26512 KB*: about 26 MB, matching our three 8 MB matrices, a sanity check on memory usage. *Major page faults 0*: nothing had to come from disk, so there is no swapping. *Minor page faults 6189*: first-touches of the matrix pages, expected and harmless. This single command told you, in one second, that the program is a CPU-or-memory-bound single-threaded job that does not swap and does not do I/O, which immediately points you toward perf stat and away from disk or system-call investigations.

> **Key Insight**
> The User-versus-System time split is the fastest triage there is. High user time means the problem is in your code (reach for perf and Cachegrind). High system time means the problem is in system calls (reach for strace). High wall time but low CPU percentage means the program is waiting, on I/O, locks, or the network (reach for vmstat and iostat). Major page faults mean swapping, which dwarfs every other concern. You can read all of this off one /usr/bin/time -v before touching a real profiler.

## 17.2 strace: The System Call Tracer

When /usr/bin/time shows high system time, the program is spending its life in the kernel, and strace tells you exactly which system calls and how often. strace intercepts every system call the program makes and logs it, with arguments, return values, and optionally timing. Its summary mode is the most useful: it counts the calls and the time spent in each.

```bash
# summary: count and time each syscall type
strace -c ./matmul_naive

# timing: how long each individual syscall took
strace -T ./matmul_naive 2>&1 | head -40

# follow a specific syscall, e.g. all mmap calls
strace -e trace=mmap ./prog
```

The summary mode (*-c*) prints a table of system calls sorted by time, with call counts and per-call averages. For matmul_naive this table is nearly empty, because the program makes almost no system calls; it allocates memory once and computes. That emptiness is itself the answer: a program with a near-empty strace summary is not system-call-bound, and you should look elsewhere. The summary mode shines on programs that are unexpectedly system-call-heavy, where it reveals, say, a million tiny read calls that should have been a few large ones, or repeated futex calls indicating lock contention.

For our project, strace is genuinely useful on Kraken-2, because Kraken-2 loads its database with memory mapping, and the mmap system calls show exactly when and how the 180 GB database is mapped into the address space.

```bash
# watch how Kraken-2 maps its database
strace -e trace=mmap,munmap,madvise \
       kraken2 --db eskape_db reads.fastq 2>&1 | head -30
```

This reveals the mmap calls that map the database file, their sizes, and any madvise hints Kraken-2 gives the kernel about access patterns. Seeing the mapping happen at the system-call level complements the Massif page-level view from Chapter 9: Massif showed the memory footprint growing as pages are touched, while strace shows the mapping calls that set up the address space in the first place. Together they give the complete picture of how the database gets into memory.

## 17.3 ltrace: The Library Call Tracer

Where strace traces calls into the kernel, *ltrace* traces calls into shared libraries. It shows which library functions the program calls, such as malloc, memcpy, or functions from libraries the program links against. It is less commonly needed than strace, because most performance problems are either in your code (profilers) or in the kernel (strace), but ltrace occasionally reveals that a program is spending surprising time in a library routine, a flood of malloc calls, or repeated expensive library functions in a loop. It works the same way as strace, with a *-c* summary mode that counts and times the library calls. Reach for it when you suspect the bottleneck is in a linked library rather than in your own code or the kernel.

## 17.4 Inspecting a Running Process Through /proc

Linux exposes a vast amount of per-process information through the */proc* filesystem, where each running process has a directory named by its process ID. For performance work the most useful file is */proc/PID/status*, which reports the process's memory usage live, while it runs, without any profiler attached.

```bash
# watch a running Kraken-2's memory in real time
watch -n1 'cat /proc/$(pgrep -n kraken2)/status | grep -E "VmRSS|VmPeak|VmSwap|VmHWM"'
```

The fields to watch are *VmRSS*, the resident set size, the physical memory the process currently occupies; *VmHWM*, the high-water mark, the peak resident size so far; *VmPeak*, the peak virtual address space; and *VmSwap*, how much has been pushed to swap. For Kraken-2 with its 180 GB mapped database, watching VmRSS climb as the database pages in, and watching VmSwap stay at zero (or alarmingly rise) tells you whether the machine has enough physical memory to hold the working set or is being forced to swap. A rising VmSwap is a red alert: the program is thrashing between memory and disk, and no CPU optimization will help until the memory pressure is resolved. This live view is something no after-the-fact profiler provides, and it is exactly the right tool for understanding the memory behavior of a long-running, memory-hungry process.

## 17.5 vmstat and iostat: The System-Wide View

The tools so far look at a single process. *vmstat* and *iostat* look at the whole system, sampled once per second, and they answer the question "is the machine itself under pressure?" which a per-process tool cannot.

```bash
vmstat 1        # memory, swap, and CPU every second
iostat -x 1     # per-device I/O statistics every second
```

vmstat prints a row every second with columns for free memory, swap in and swap out, I/O blocks in and out, context switches and interrupts, and the CPU time split across user, system, idle, and I/O wait. The two columns that matter most for triage are the swap columns and the I/O-wait CPU column. Nonzero swap-in and swap-out mean the system is paging to disk, the single most destructive performance problem and one that per-process profilers do not surface clearly. A high I/O-wait percentage means the CPU is idle waiting for disk, which means the bottleneck is storage, not computation. iostat goes deeper on the storage side, showing per-device utilization and the average wait time for I/O requests, so you can see if a particular disk is saturated. For Kraken-2 reading a large input file, or for Dorado streaming signal data from disk, iostat reveals whether the storage is keeping up or is the bottleneck starving the GPU, which ties directly back to the GPU-idle gaps from Chapter 12.

## 17.6 numactl: NUMA Awareness

On a multi-socket server, and the LUNA cluster nodes and possibly the sauron machine qualify, memory is not uniform. Each CPU socket has its own attached memory, and accessing memory attached to another socket is slower because it crosses an inter-socket link. This is NUMA, Non-Uniform Memory Access, and ignoring it can silently halve your memory performance. *numactl* is the tool for seeing the NUMA topology and for binding a program's memory and threads to the same socket so they stay local.

```bash
# show the NUMA topology: nodes, their CPUs, and their memory
numactl --hardware

# bind a program to run on node 0 with its memory on node 0
numactl --cpunodebind=0 --membind=0 \
        kraken2 --db eskape_db reads.fastq
```

The *--hardware* view lists the NUMA nodes, which CPUs belong to each, and how much memory each has, along with a distance matrix showing the relative cost of cross-node access. The binding options pin a program so that its threads and its memory live on the same node, avoiding the cross-socket penalty. For Kraken-2 this matters because the 180 GB database, if allocated without NUMA awareness, can end up split across nodes so that threads on one socket constantly reach across to memory on another, a slowdown that shows up in perf mem from Chapter 5 as remote-DRAM accesses. Binding with numactl, or interleaving the database across nodes deliberately, can recover that lost performance. On a single-socket machine NUMA does not apply,

but the moment you run on a multi-socket cluster node, checking numactl --hardware and considering binding is part of responsible profiling.

## 17.7 The Complete Diagnostic Workflow

Now let me assemble all of this, including the heavy tools from the earlier batches, into the single diagnostic workflow I follow on any performance problem. This is the practical synthesis: the order in which you reach for tools, from cheapest and widest to most expensive and narrowest. Follow this order and you rarely waste time pointing a deep tool at the wrong problem.

- **Step 1, /usr/bin/time -v:** is this CPU-bound, memory-bound, system-call-bound, or waiting? Read the user-versus-system split, the CPU percentage, the major page faults, and the peak resident size. One second, no setup.
- **Step 2, vmstat 1:** is the system swapping or waiting on I/O? If swap is active or I/O-wait is high, the problem is memory pressure or storage, and you must fix that before any CPU profiling means anything.
- **Step 3, perf stat:** what is the IPC and the cache-miss rate? This is the first real CPU diagnostic, separating compute-bound from memory-bound, as in Chapter 3.
- **Step 4, if cache misses are high, perf record plus Cachegrind:** where exactly are the misses, attributed to function and line, as in Chapters 4 and 7.
- **Step 5, if it is system-call-heavy, strace -c:** which system calls dominate, as in this chapter.
- **Step 6, if a GPU is involved, Nsight Systems then Nsight Compute:** survey the timeline for idle gaps and transfer overhead, then dissect the dominant kernel, as in Chapters 12 and 13.

> **How This Connects to Your Project**
>
> This workflow is exactly how you should approach both halves of the Nanopore pipeline, and following it would have shaped the whole investigation. For Kraken-2: /usr/bin/time -v confirms it is CPU-and-memory-bound, vmstat confirms whether the 180 GB database fits in RAM or forces swapping (a critical early check, since swapping would dominate everything), perf stat confirms the low IPC and high cache-miss rate, perf record and Cachegrind localize the misses to the hash lookup, and strace reveals the mmap-based database loading. Five cheap-to-moderate steps and you have the entire Kraken-2 story.
>
> For Dorado: /usr/bin/time -v shows the host-side picture, iostat reveals whether disk I/O feeding the signal data is starving the GPU (connecting to the timeline gaps from Chapter 12), and then Nsight Systems and Nsight Compute do the GPU-side analysis. The system tools are what tell you whether Dorado's bottleneck is even on the GPU at all, or whether it is upstream in data loading. Starting with these cheap tools prevents the classic mistake of deeply profiling a GPU kernel when the real problem was that the disk could not feed it fast enough.

### Common Pitfall

Reaching for perf and Nsight before running the cheap system tools. If the program is swapping or I/O-bound, the CPU profilers will mislead you. Triage with /usr/bin/time -v and vmstat first.

Using the shell builtin time instead of /usr/bin/time -v and missing all the memory and page-fault information. The full binary with -v is the one that triages.

Ignoring NUMA on a multi-socket machine. Cross-socket memory access can halve performance silently. Check numactl --hardware on cluster nodes and consider binding.

Missing a swapping problem because per-process profilers do not surface it clearly. vmstat's swap columns are the definitive check, and a swapping program cannot be fixed by any CPU optimization until the memory pressure is resolved.

### What We Just Learned

The Linux system tools are the cheap, wide first layer of any investigation, run before the heavy profilers. They reveal problems profilers miss: swapping, I/O waits, system-call floods, NUMA penalties.

/usr/bin/time -v triages in one command: the user-versus-system time split, CPU percentage, major page faults, and peak resident size tell you whether the problem is your code, the kernel, waiting, or swapping.

strace traces system calls (use -c for the summary) and reveals Kraken-2's mmap-based database loading; ltrace traces library calls for the rarer case of a library-bound bottleneck.

/proc/PID/status shows a running process's live memory (VmRSS, VmHWM, VmPeak, VmSwap), the right tool for watching a long-running memory-hungry process like Kraken-2 and catching swap thrashing early.

vmstat and iostat give the system-wide view: swapping activity, I/O wait, and per-device storage saturation, which per-process tools cannot show. numactl reveals NUMA topology and binds memory and threads to the same socket on multi-socket machines.

The complete diagnostic workflow goes cheap-to-expensive: /usr/bin/time -v, then vmstat, then perf stat, then perf record plus Cachegrind for memory or strace for system calls, then Nsight if a GPU is involved. Following this order prevents pointing deep tools at the wrong problem.

### Before You Move On

You now have the triage layer that frames every investigation, and the complete cheap-to-expensive workflow that ties all four batches of tools into a single practice. This is the connective tissue that makes the heavy tools effective rather than misapplied.

The next chapter is LIKWID, a specialist tool for hardware-counter expertise and, crucially, for measuring the machine's actual peak memory bandwidth, the honest denominator you need for the roofline. It is how you turn "the kernel achieves some bandwidth" into "the kernel achieves X percent of the measured peak," which is exactly the kind of grounded, fraction-of-peak statement that makes the upcoming report credible rather than hand-wavy.

CHAPTER 18

# LIKWID and Counter Expertise

There is a specific gap in everything we have learned, and this chapter fills it. To draw a roofline, the Chapter 5 framework that answers how-much-faster, you need two machine numbers: the peak compute rate and the peak memory bandwidth. So far we have used the spec-sheet values, the marketing numbers, and I warned you back in Chapter 5 that those overstate what the machine actually delivers. The real, sustained bandwidth your machine achieves under a realistic access pattern is always lower than the spec, and reporting your kernel's efficiency against the spec number understates how good you really are. LIKWID is the tool that measures the machine's true peak directly, and it is also a clean, scriptable way to read hardware counters grouped into ready-made bundles. It is the tool that turns "the kernel achieves some bandwidth" into "the kernel achieves X percent of the measured peak," which is the grounded, fraction-of-peak statement that makes a report credible.

LIKWID stands for Like I Knew What I am Doing, a name that captures its spirit: it packages hardware-counter expertise into commands you can run without being a microarchitecture expert. Where perf gives you raw events and leaves the grouping and interpretation to you, and VTune wraps everything in a GUI, LIKWID sits in between: command-line, scriptable, with curated event groups that bundle the right counters for a given question and compute the derived metrics for you. It is the tool of choice in the high-performance-computing community precisely because it is reproducible, scriptable, and oriented toward the multi-core and NUMA reality of cluster machines like LUNA.

## 18.1 The LIKWID Tool Suite

LIKWID is a collection of small tools, each focused on one job. Four matter for our purposes.

- **likwid-topology:** shows the CPU, cache, and NUMA topology in detail, the physical layout of cores, caches, and memory nodes. It is the LIKWID complement to numactl --hardware, with more detail on the cache hierarchy.
- **likwid-perfctr:** the main event, a hardware-counter measurement tool that reads counters per core or per socket, organized into named event groups, and computes derived metrics. This is the LIKWID equivalent of perf stat, but with curated groups and per-core attribution.
- **likwid-bench:** a microbenchmark suite that measures the machine's actual sustained memory bandwidth and other low-level capabilities. This is the tool that gives you the honest roofline denominator.
- **likwid-pin:** pins threads to specific cores to avoid the migration effects that muddy measurements. Pinning ensures that when you measure a thread's behavior, the OS does not move it to a cold core mid-measurement.

Installation is through the package manager on most distributions, or built from source for the latest version. LIKWID needs access to the hardware counters, which on Linux means either the same perf_event interface (the perf_event_paranoid setting from Chapter 3 applies) or LIKWID's own kernel access daemon for finer control.

```bash
# Ubuntu/Debian
sudo apt install likwid

# verify and view the machine topology
likwid-topology -g        # -g draws an ASCII diagram of the topology
```

## 18.2 likwid-topology: Know Your Machine

Before measuring anything, you need to know the machine's physical layout, because counters are attributed per core and per socket, and the roofline depends on per-socket bandwidth and total core count. likwid-topology lays this out clearly: the number of sockets, the cores per socket, the threads per core (hyperthreading), the size and sharing of each cache level, and the NUMA node structure. The *-g* option draws an ASCII diagram that makes the hierarchy visible at a glance, showing which cores share an L2, which share the L3, and how the NUMA nodes map onto sockets. For the lab Dell Precision with its Xeon W-2123, this tells you the four cores, their private L1 and L2, the shared L3, and the single NUMA node. For a LUNA cluster node it would reveal the multi-socket, multi-NUMA structure that shapes how you should bind and measure. Knowing the topology is the prerequisite for interpreting per-core counter output correctly.

## 18.3 likwid-perfctr: Curated Counter Groups

The headline tool reads counters into named groups. Instead of remembering that the L3 miss rate requires a specific combination of cache-reference and cache-miss events, you ask for the CACHE group and LIKWID measures the right counters and computes the miss rates for you. You select a group with *-g* and the cores to measure with *-C*.

```bash
# measure the CACHE group on core 0
likwid-perfctr -C 0 -g CACHE ./matmul_naive

# measure memory bandwidth group
likwid-perfctr -C 0 -g MEM ./matmul_naive

# measure double-precision FLOPS
likwid-perfctr -C 0 -g FLOPS_DP ./matmul_naive

# list all available groups on this machine
likwid-perfctr -a
```

The CACHE group produces a table with the L1, L2, and L3 miss rates and the data volume at each level, all computed and labeled, no manual arithmetic. For matmul_naive it will show the high

miss rates we have seen throughout, now attributed per core and presented as a clean derived table. The value over raw perf is that the group already knows which counters compose the miss rate and does the division for you, and the output is laid out for direct reading. The *-a* flag lists every available group, and the set is rich: CACHE, MEM, FLOPS_DP, FLOPS_SP, BRANCH, DATA, L2, L3, and more, each a curated bundle for a specific question. Choosing the right group is choosing your question; LIKWID handles the counter mechanics.

The per-core targeting via *-C* matters on parallel programs. You can measure a specific core, a range of cores, or all cores, and the output is broken down per core so you can see imbalance. Combined with likwid-pin to keep threads on their assigned cores, this gives clean per-core measurements of a parallel workload, which is how you diagnose whether all cores are equally busy or whether some are starved, the LIKWID approach to the load-imbalance question that VTune's Threading analysis and perf c2c also address from their own angles.

## 18.4 likwid-bench: The Honest Roofline Denominator

Now the tool that completes the roofline. likwid-bench runs a suite of carefully written microbenchmarks that measure what the machine actually delivers, as opposed to its theoretical spec. The most important is the STREAM benchmark, the long-standing standard for measuring sustained memory bandwidth, which LIKWID implements with proper care for alignment, NUMA placement, and access pattern.

```bash
# measure sustained memory bandwidth with a STREAM-style benchmark
likwid-bench -t stream -w S0:512MB

# the -t selects the benchmark kernel (stream = read+write triad)
# the -w specifies the workgroup: socket 0, 512 MB working set
```

This reports the achieved bandwidth in GB/s for a working set large enough to exceed all caches and genuinely hit DRAM. The number it returns is typically 60 to 80 percent of the spec-sheet bandwidth, because real access patterns never reach the theoretical maximum that assumes perfect everything. *This measured number is the correct y-axis denominator for your roofline.* When you then report that your kernel achieves, say, 200 GB/s, you compare it against the measured sustainable peak rather than the spec, and you get an honest efficiency figure. Reporting against the spec number makes your kernel look worse than it is; reporting against the measured sustainable peak is both fair and defensible, and it is the kind of grounded number that distinguishes a careful report from a careless one.

likwid-bench has many kernels beyond stream: copy, triad, load, store, and various arithmetic kernels that measure peak FLOPS for different operation mixes. Between the bandwidth measurement and a FLOPS measurement, you have both axes of the roofline, both grounded in what the machine actually does rather than what its datasheet claims. This is the rigorous way to build the roofline that Chapter 5 introduced and that Nsight Compute drew for the GPU: on the CPU side, you measure the ceilings yourself with likwid-bench, and you place your kernel against those measured ceilings.

**And Here Is Where It Gets Beautiful**

The roofline finally closes here. Chapter 5 gave you the model. Chapter 13 showed Nsight drawing it automatically for the GPU. But for the CPU, you needed honest peak numbers, and the spec sheet lies. likwid-bench measures the true sustained bandwidth and peak FLOPS of your actual machine, so you can place your kernel on a roofline whose ceilings are real. Now "the kernel achieves X percent of peak" means X percent of what the machine genuinely delivers, not X percent of a marketing number. That honesty is what makes a fraction-of-peak claim worth putting in front of a professor, and it is the difference between a roofline that informs and a roofline that flatters.

## 18.5 Why Counter Expertise Matters for the Report

Let me connect this directly to the deliverable, because LIKWID's value is most visible in what it lets the report say. A weak report says "Kraken-2 is memory-bound" and "Dorado's attention kernel is slow." A strong report says "Kraken-2 achieves X percent of the machine's measured peak DRAM bandwidth, confirming it is memory-bandwidth-bound" and "Dorado's attention kernel achieves Y percent of the T4's peak compute." The difference is the fraction-of-peak framing, and the fraction-of-peak framing requires knowing the true peak, which requires either Nsight's automatic GPU roofline or, on the CPU, LIKWID's measured ceilings.

The discipline LIKWID instills is to always express performance as a fraction of a measured ceiling, never as a raw number in isolation. A raw bandwidth of 200 GB/s means nothing without the peak; against a 250 GB/s measured peak it is excellent, against a 400 GB/s peak it is mediocre. The fraction is the meaningful quantity, and it is also the one that tells you when to stop: a kernel at 85 percent of measured peak has little headroom and you should move on, while a kernel at 20 percent has a 4x opportunity. This is the quantitative maturity that separates a profiling report that merely describes from one that evaluates and recommends, and LIKWID is the tool that makes it routine on the CPU side.

**How This Connects to Your Project**

For the Kraken-2 report page, the workflow is: run likwid-bench stream on the lab machine to get the measured peak DRAM bandwidth, run likwid-perfctr with the MEM group on Kraken-2 to get its achieved bandwidth, and report the ratio as the fraction of peak. A Kraken-2 that achieves a high fraction of peak bandwidth while still being slow is bandwidth-bound and needs less traffic; one that achieves a low fraction despite a high miss rate is latency-bound, waiting on scattered accesses rather than saturating the bus, which is the random-probe pattern the Hot-K-mer cache targets. This distinction, the same one VTune's TMA draws between DRAM Bandwidth Bound and DRAM Latency Bound, is confirmed quantitatively by comparing achieved bandwidth to measured peak.

For Dorado, Nsight Compute already draws the roofline against the GPU's ceilings automatically, so you do not need LIKWID on the GPU side. But the same fraction-of-peak discipline applies: report the attention kernel's DRAM throughput as a fraction of the T4's measured peak bandwidth, and its compute as a fraction of peak FLOPS, so the report states not just what the kernel does but how close to the ceiling it is and therefore how much headroom remains. LIKWID's lesson, express everything as a fraction of a measured peak, is what makes both pages of the report quantitatively honest.

### Common Pitfall

Building a roofline against spec-sheet bandwidth instead of measured bandwidth. The spec overstates the real ceiling, so your kernel looks less efficient than it is. Measure the true peak with likwid-bench.

Reporting raw bandwidth or FLOPS numbers without a ceiling for context. A raw number is meaningless; the fraction of measured peak is the quantity that informs and that tells you when to stop optimizing.

Measuring per-core counters without pinning threads, so the OS migrates a thread mid-measurement and the per-core attribution is muddied. Use likwid-pin for clean parallel measurements.

Ignoring the machine topology before interpreting counters. Per-socket bandwidth and core counts depend on the layout that likwid-topology reveals; misreading the topology means misreading the counters.

### What We Just Learned

LIKWID packages hardware-counter expertise into scriptable command-line tools with curated event groups, sitting between perf's raw counters and VTune's GUI. It is the high-performance-computing community's tool of choice for reproducible, multi-core, NUMA-aware measurement.

likwid-topology reveals the physical layout (sockets, cores, cache sharing, NUMA nodes), the prerequisite for interpreting per-core counters and for building the roofline correctly.

likwid-perfctr reads counters into named groups (CACHE, MEM, FLOPS_DP, and more) per core or socket and computes the derived metrics for you, the curated-group equivalent of perf stat. likwid-pin keeps threads on their cores for clean parallel measurement.

likwid-bench measures the machine's actual sustained bandwidth (via STREAM) and peak FLOPS, which run 60 to 80 percent of spec. These measured numbers are the honest denominators for the CPU roofline.

Always express performance as a fraction of a measured ceiling, never as a raw number. The fraction is what informs and what tells you when to stop: high fraction means little headroom, low fraction means opportunity.

For Kraken-2, comparing achieved bandwidth to measured peak distinguishes bandwidth-bound (high fraction, reduce traffic) from latency-bound (low fraction despite high miss rate, improve locality with caching), the same distinction TMA draws. For Dorado, Nsight draws the GPU roofline automatically, but the fraction-of-peak discipline applies equally.

### Before You Move On

You now have every tool the book offers and the quantitative discipline to use them honestly: express performance as a fraction of a measured peak. Between the four batches you can profile any program on either side of the PCIe bus and ground every claim in a measured ceiling.

The next chapter is the one everything has been building toward: how to actually write the two-page profile report for Prof. Paul. It is not a tool chapter. It is about turning all the numbers you can now produce into a clear, structured, defensible document that identifies the bottlenecks and recommends the right fixes. Every cell of the report tables, you now know how to fill. The next chapter is about assembling them into something worth reading.

CHAPTER 19

# Writing the 2-Page Report

Everything in this book has been building toward this chapter. Eighteen chapters of tools, and now we turn all of it into the actual deliverable: a two-page profile report for Prof. Paul. This is not a tool chapter. It is a writing chapter, and it is arguably the most important one, because a profiling investigation that does not end in a clear report is an investigation nobody can act on. You can produce every number in this book, but if you cannot assemble those numbers into a document that identifies the bottlenecks and recommends the right fixes, the work is invisible. The skill of writing a good profile report is distinct from the skill of profiling, and it is the one that makes the profiling matter.

Let me name the wrong instinct first, because almost every student's first report makes this mistake. The instinct is to paste the tool output. You run perf stat and Nsight Compute, you copy the tables into a document, and you call it a report. This is not a report; it is a data dump. The reader cannot tell what matters, what the numbers mean, or what to do. A report is not the tool output. A report is the *interpretation* of the tool output: the bottleneck the numbers reveal, expressed as fractions of peak, with a recommendation that follows from the diagnosis. The tables are evidence, not the argument. This chapter is about building the argument.

## 19.1 What a Good Profile Report Contains

A profile report has a structure, and the structure is the same regardless of what you profiled. Each section answers a question the reader will ask, in the order they will ask it. Master this structure once and every report you write for the rest of your career follows it.

- **Executive summary:** two or three sentences stating the headline findings. The reader who reads only this should know the bottlenecks and the recommendation. Example: "Dorado is memory-bandwidth-bound on the T4, with the attention kernel achieving 58 percent of peak bandwidth. Kraken-2's k-mer lookup has a 78 percent L3 miss rate, indicating a latency-bound random-access pattern. Both are addressable by caching." That is the whole report in three sentences, and a busy professor may read only this.
- **System configuration:** the exact hardware and software, so the results are reproducible and interpretable. GPU model, CPU model, RAM, OS, driver and tool versions, and the exact commands used. Without this, the numbers float free of any context.
- **Methodology:** how you ran the profiler and on what workload. Which input, how many runs, whether you took the median, whether you profiled steady state or included startup. This is what lets the reader trust the numbers.
- **Results as structured tables:** the evidence, organized. For the GPU side, a per-kernel table; for the CPU side, a counter table and a hotspot list. Numbers presented as fractions of peak wherever possible.

- **Bottleneck identification:** the interpretation. What the numbers mean. This is where you state, in prose, the diagnosis the tables support: memory-bound versus compute-bound versus latency-bound, and why.
- **Recommendations:** what to optimize and why, following directly from the bottleneck. Each recommendation should trace to a specific number in the results. Never recommend something the data does not support.

> **Key Insight**
> The structure is a funnel from conclusion to evidence to action. The executive summary states the conclusion for the impatient reader. The tables provide the evidence for the skeptical reader. The recommendations give the action for the reader who wants to do something. A reader can stop at any depth and still get value. This is the opposite of a data dump, which forces every reader to do the interpretation themselves. Write conclusion-first, evidence-second, and your report serves every reader.

## 19.2 The Exact Two-Page Structure for the Nanopore Project

For our specific project, the two pages divide cleanly by pipeline stage. Page one is Dorado, the GPU side. Page two is Kraken-2, the CPU side. Each page follows the structure above, scaled to fit, and each is anchored by one or two tables and a short interpretation.

### Page 1: Dorado (GPU)

Page one opens with the Dorado portion of the executive summary and the GPU system configuration, then presents the evidence from the Nsight tools. The centerpiece is a per-kernel table built from Nsight Systems and Nsight Compute: each dominant kernel with its duration, its SM occupancy, its DRAM throughput as a fraction of the T4 peak, and its L2 hit rate. Below the table, a statement of the roofline placement from Nsight Compute: which kernels are memory-bound, which are compute-bound, and the headroom to the ceiling. Then a one-sentence recommendation, such as reducing global-memory traffic on the memory-bound attention kernel via the Signal-to-Base cache. Optionally a small timeline screenshot from Nsight Systems showing the CPU-GPU coordination and any idle gaps.

### Page 2: Kraken-2 (CPU)

Page two opens with the Kraken-2 portion of the executive summary and the CPU system configuration, then presents the perf and Cachegrind evidence. The centerpiece is a perf stat table: cycles, instructions, IPC, branch miss rate, and LLC miss rate, with the derived ratios computed. Alongside it, a hotspot list from perf record showing the top functions by self time, confirming the hash lookup dominates. The LLC miss rate comes from Cachegrind on WSL2 (with the footnote that it is simulated because the hardware counters are unavailable under Hyper-V) or from perf directly on a machine where the counters work. Then the bottleneck interpretation: memory-bound, and specifically latency-bound from random hash-table probing, supported by the low IPC and high miss rate. Then the recommendation: the Hot-K-mer LRU cache to make the hot accesses local.

## 19.3 A Complete Worked Example: matmul_naive versus matmul_tiled

Before templating the real report, let me write a complete miniature report comparing the two CPU matmul kernels, because it shows the structure filled in with real numbers and real interpretation, at a scale you can absorb in one reading. This is the report you would write if Prof. Paul had asked you to profile the matmul optimization.

### Executive summary (worked example)

The naive triple-loop matmul is memory-bandwidth-bound, achieving an IPC of 0.32 and a 46 percent LLC miss rate due to stride-n column access to matrix B. Reordering the loops to i-k-j (the ikj variant) restores stride-1 access, raising IPC to 2.10 and dropping the LLC miss rate to under 6 percent, a 10x speedup with no algorithmic change. Tiling the loops yields a further modest gain by improving temporal locality. The optimization confirms that the workload's true bottleneck was cache behavior, not arithmetic.

### System configuration (worked example)

Intel Core i7-10750H, 6 cores at 2.6 GHz base, 12 MB L3; 16 GB DDR4; Ubuntu 22.04; gcc 11.4 at -O2 -g; perf from linux-tools 5.19; Valgrind 3.18. Matrix dimension N equals 1024, double precision. Each result is the median of five runs.

### Results table (worked example)

output

```
Variant       Time(s)   IPC    LLC miss rate   D1 miss rate (cg)
-----------   -------   ----   -------------   -----------------
matmul_naive    7.84    0.32       46.1%            33.5%
matmul_ikj      0.75    2.10        5.7%             4.2%
matmul_tiled    0.68    2.35        2.9%             2.1%
```

Bottleneck identification: the naive variant's low IPC and high miss rate identify a memory-bandwidth bottleneck, confirmed by Cachegrind's 33.5 percent D1 read miss rate localized to the inner-loop access of B. The ikj reordering eliminates the stride-n access; the dramatic IPC rise and miss-rate fall confirm the bottleneck was the access pattern. Tiling further improves temporal locality, approaching the arithmetic-bound regime where IPC plateaus. Recommendation: use the ikj ordering as the baseline for any further work; tiling provides marginal additional benefit and is worthwhile only if the inner kernel is reused heavily. No arithmetic optimization is warranted, since the workload is not compute-bound until the cache behavior is fixed.

Notice what that worked example does. Every number traces to a tool: IPC and LLC miss rate from perf stat, D1 miss rate from Cachegrind, all from chapters you have read. Every interpretation follows from a number. The recommendation follows from the interpretation. There is no orphan data and no unsupported claim. That is the standard for the real report.

## 19.4 Templates for the Real Report

Now the templates for the actual Dorado and Kraken-2 pages. These are the skeletons you fill with the numbers your tools produce. I write them as the tables and the prose frames, with placeholders where the measured values go.

### Dorado results table template

output

```
Kernel              Time(%)  Dur(ms)  Occupancy  DRAM(% peak)  L2 hit
-----------------   -------  -------  ---------  ------------  ------
attention_gemm        __       __        __%         __%        __%
conv_stem             __       __        __%         __%        __%
ctc_decode            __       __        __%         __%        __%
layernorm/softmax     __       __        __%         __%        __%
```

Fill the Time and Duration columns from nsys stats cuda_gpu_kern_sum, the Occupancy and DRAM and L2 columns from ncu on each dominant kernel. The DRAM column as a fraction of the T4's measured peak, per the LIKWID discipline from Chapter 18, applied here through Nsight's roofline. Then write the interpretation prose: which kernel dominates, whether it is memory-bound or compute-bound from its Speed Of Light and roofline placement, and the recommendation that follows.

### Kraken-2 results table template

output

```
Metric                       Value      Target    Interpretation
--------------------------   --------   -------   ----------------------
IPC                            __         > 2.0    low => memory-bound
LLC miss rate                  __%        < 10%    high => DRAM-bound
Branch miss rate               __%        < 1%     ~ low, not the issue
Hot function (perf record)   <name>       ---      hash lookup expected
LLC miss rate of hot fn (cg)   __%        ---      simulated on WSL2
```

Fill IPC and branch miss rate from perf stat, the hot function from perf record, and the LLC miss rate from Cachegrind on WSL2 (simulated, footnoted) or perf where available. Then the interpretation: the low IPC and high miss rate identify a memory bottleneck; the perf record hotspot confirms the hash lookup is where time and misses concentrate; the recommendation is the Hot-K-mer cache, justified because the bottleneck is the random-access latency of the lookup, not the arithmetic.

## 19.5 How to Justify Numbers

The single most important habit in report writing, and the one that elevates a report from descriptive to evaluative, is to never present a raw number alone. Every number should be accompanied by what it should be (a target or a peak) and what it means (the interpretation). This is the fraction-of-peak discipline from Chapter 18 applied to the entire report.

Concretely: do not write "the kernel achieved 185 GB/s." Write "the kernel achieved 185 GB/s, which is 58 percent of the T4's measured peak bandwidth of 320 GB/s, indicating it is moderately bandwidth-bound with roughly 40 percent headroom." The raw number alone tells the reader

nothing; the same number against the peak, with the interpretation, tells them the kernel's efficiency, its bottleneck class, and its headroom in a single sentence. Do this for every number. The IPC against its target, the miss rate against its threshold, the bandwidth against the measured peak, the occupancy against the maximum. A report where every number is grounded this way reads as authoritative because it demonstrates that you understand not just what the tools reported but what the reports mean.

The corollary is to convert tool output into percentages of peak wherever you can, because percentages are interpretable across machines and raw numbers are not. A reader who does not know your specific GPU cannot judge 185 GB/s, but anyone can judge 58 percent of peak. The percentage carries the meaning; the raw number carries only the measurement. State both, lead with the percentage in the interpretation, and your report communicates regardless of the reader's familiarity with your specific hardware.

**How This Connects to Your Project**

This chapter is the deliverable itself. The two-page report for Prof. Paul is page one Dorado, page two Kraken-2, each following the funnel structure: executive summary, configuration, methodology, results table, bottleneck interpretation, recommendation. You now know how to fill every cell, from the Nsight kernel table to the perf stat counters to the Cachegrind LLC simulation, and how to ground every number as a fraction of peak.

The recommendations write themselves once the bottlenecks are identified, because the whole book has been pointing at them: Dorado's memory-bound attention kernel wants the Signal-to-Base cache, and Kraken-2's latency-bound hash lookup wants the Hot-K-mer LRU cache. But the discipline of this chapter is that you do not state those recommendations until the numbers support them. You let perf and Cachegrind and Nsight establish memory-boundness first, then recommend the traffic-reduction or locality fix as the conclusion the evidence demands. A report that recommends the right fix for the wrong (or unstated) reason is luck; a report that recommends it as the documented consequence of measured bottlenecks is engineering.

**Common Pitfall**

Pasting tool output and calling it a report. The output is evidence; the report is the interpretation. The reader needs the conclusion, not the raw tables alone.

Presenting raw numbers without a peak or target for context. 185 GB/s means nothing; 58 percent of measured peak means everything. Ground every number.

Omitting the system configuration, so the numbers cannot be reproduced or interpreted. Always state the exact hardware, software versions, and commands.

Recommending an optimization the data does not support, or stating the recommendation before establishing the bottleneck. Let the numbers establish the diagnosis, then recommend the fix the diagnosis demands.

### What We Just Learned

A report is the interpretation of tool output, not the output itself. Pasting tables is a data dump; building the argument from the numbers is a report.

The structure is a funnel: executive summary (conclusion), system configuration and methodology (trust), results tables (evidence), bottleneck identification (interpretation), recommendations (action). A reader can stop at any depth and gain value.

For the Nanopore project, page one is Dorado (a per-kernel table from Nsight, roofline placement, a traffic-reduction recommendation) and page two is Kraken-2 (a perf stat counter table, a perf record hotspot list, a Cachegrind LLC rate, a locality recommendation).

The worked matmul report shows the standard: every number traces to a tool, every interpretation follows from a number, every recommendation follows from the interpretation. No orphan data, no unsupported claims.

Never present a raw number alone. Pair it with its target or peak and its interpretation: not 185 GB/s, but 58 percent of measured peak, moderately bandwidth-bound, 40 percent headroom. Convert to fractions of peak so the meaning carries across machines.

The recommendations follow from the bottlenecks: Signal-to-Base caching for Dorado's memory-bound attention, the Hot-K-mer cache for Kraken-2's latency-bound lookup. State them only after the numbers establish the diagnosis.

### Before You Move On

You can now write the report, which is the entire point of the book. You can produce every number and assemble it into a structured, grounded, defensible two-page document that diagnoses both halves of the pipeline and recommends the right fixes.

The final chapter steps back to the meta-skill: given any future performance question, which tool do you reach for? It is the decision framework that ties all twenty chapters into a single mental flowchart, plus the profiling discipline (the dos and don'ts) and the optimization loop. It closes with the appendices: the complete benchmark source, the command reference sheet, and the catalog of common perf output patterns. It is the chapter you will return to long after the rest fades, as the map of the whole territory.

CHAPTER 20

# When to Use Which Tool

We arrive at the last chapter, and it is the one you will return to long after the details of any single tool fade. The book gave you a toolbox of perhaps a dozen profilers across the CPU and the GPU. The meta-skill, the one that makes the toolbox useful, is knowing which tool to reach for given a question. This chapter is that decision framework, distilled into a flowchart you can hold in your head, followed by the profiling discipline that keeps you honest and the optimization loop that structures the whole activity. Then three appendices: the complete benchmark source, a command reference sheet, and a catalog of the perf output patterns you will see again and again. This is the map of the whole territory.

## 20.1 The Tool Selection Decision Tree

Here is the decision tree, the single most useful artifact in the book for daily work. Read it as a series of questions, each answered by reaching for a specific tool. Memorize the shape and you will never again stare at a slow program wondering where to begin.

output

```
Question                                  -> Tool
----------------------------------------     ------------------------
Quick sanity check, CPU or memory bound?  -> /usr/bin/time -v
Is the system swapping or I/O bound?       -> vmstat 1, iostat -x 1
What is the IPC and cache miss rate?       -> perf stat
Which function is hot?                      -> perf record + report
Which instruction inside the hot function? -> perf annotate
Where do the cache misses originate?        -> perf record -e cache-misses
Exact, deterministic cache miss counts?     -> cachegrind
LLC miss data on WSL2 (perf unsupported)?   -> cachegrind
Who calls the hot function, and how often?  -> callgrind
Is there a memory bug (leak, overflow)?     -> memcheck
How does heap memory grow over time?        -> massif
Is there a data race in threaded code?      -> helgrind / drd
Guided why-analysis via Top-Down?           -> vtune (uarch-exploration)
Which data structure causes the misses?     -> vtune memory-access
Measured peak bandwidth for the roofline?   -> likwid-bench
Curated per-core counter groups?            -> likwid-perfctr
GPU system timeline, kernel order, gaps?    -> nsight systems (nsys)
GPU single-kernel deep dive, why slow?      -> nsight compute (ncu)
Which syscalls dominate?                    -> strace -c
Quick single-threaded hotspot, no root?     -> gprof
```

The tree has a structure worth naming. The top entries are cheap and wide, the system tools that triage. The middle is the CPU profiling core, perf and Valgrind, ordered from the what question to the where question to the exact-counts question. The VTune and LIKWID entries are the specialized CPU lenses, Top-Down interpretation and measured-peak grounding. The Nsight entries are the GPU half, survey then microscope. And gprof sits at the bottom as the fallback for the constrained case. The whole book, twenty chapters, collapses into this one table, and the table

is the thing to internalize.

## 20.2 The Profiling Discipline: Dos and Donts

Beyond knowing which tool, there is a discipline that separates profiling that produces trustworthy results from profiling that produces garbage. These are the rules I follow without exception, learned mostly by violating them and paying for it.

### The donts

- **Never profile a debug build.** A -O0 build is a different program with different cache behavior, different inlining, different everything. Always profile -O2 or -O3, the optimization level you actually ship. The one exception is Memcheck, where -O0 gives clearer error attribution, but that is correctness, not performance.
- **Never report a single run.** A single run is noise. Take the median of at least five runs, and report the variation. perf stat -r 5 does this for you. A number without a sense of its variance is not a measurement.
- **Never ignore the profiling overhead itself.** Cachegrind is 50x slower; the program it measures is not the program that runs in production. Sampling has skid. Instrumentation distorts. Know the overhead of the tool you are using and account for it in your conclusions.
- **Never optimize what the profiler does not confirm is hot.** This is the cardinal sin from Chapter 1, restated as a rule. Your intuition about the bottleneck is probably wrong. Measure, confirm, then optimize. The radix-sort tragedy is always one guess away.

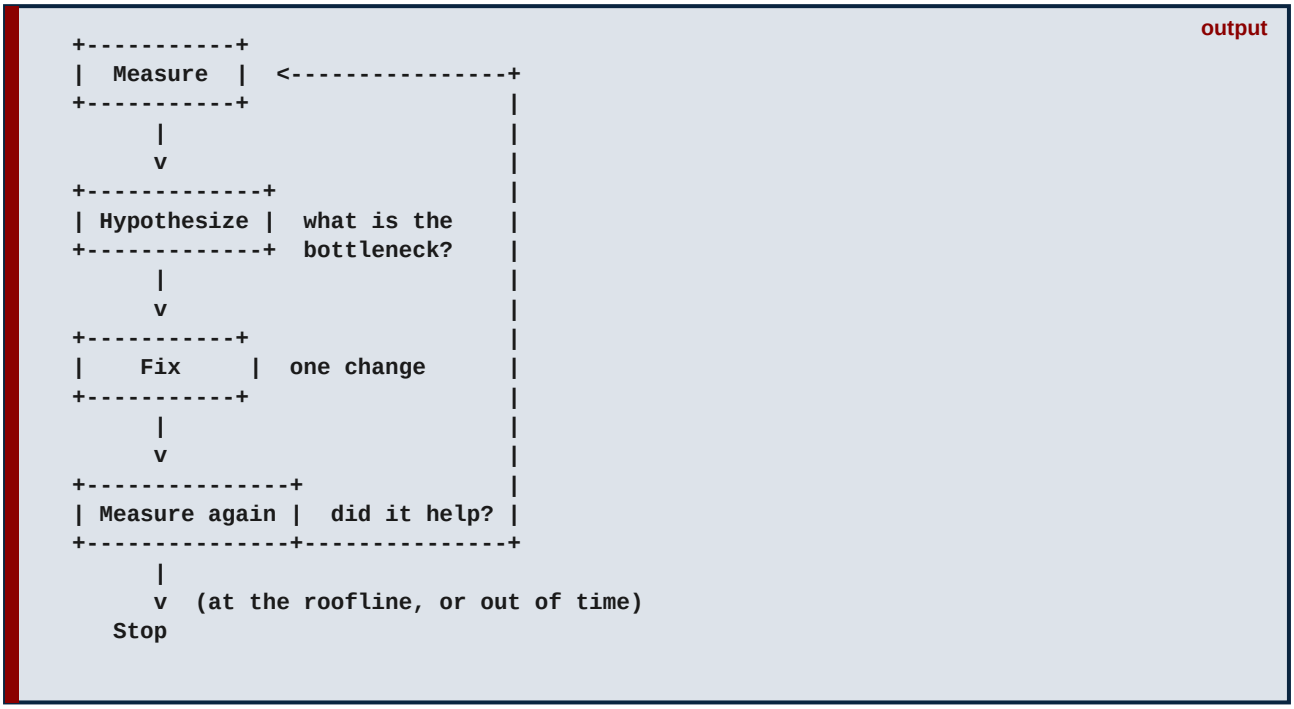
### The dos

- **Always triage cheap-to-expensive.** /usr/bin/time -v and vmstat before perf, perf before Cachegrind, Nsight Systems before Nsight Compute. Do not point an expensive narrow tool at a problem a cheap wide tool would have revealed.
- **Always express performance as a fraction of a measured peak.** The LIKWID discipline from Chapter 18. A raw number is meaningless; a fraction of peak is interpretable and tells you when to stop.
- **Always compile with -g (or -lineinfo for CUDA).** Debug info costs nothing at -O2 and lets every tool map results to source lines. The number of times I have had to re-run a profile because I forgot -g is embarrassing.
- **Always form a hypothesis before measuring.** Predict what the profiler will show, then check. Calibrating your predictions against reality is how you build the intuition that makes you fast. The book taught you to predict naive matmul's cache behavior before measuring it; do that always.

## 20.3 The Optimization Loop

Profiling is not a one-shot activity. It is a loop, and the loop has a definite shape that you run until you hit the roofline or run out of time. The shape is: measure, hypothesize, fix, measure again, repeat.

output




Watch the loop run on our matmul, because it is the whole book in one example. Round one: perf stat says low IPC, high cache misses. Hypothesis: the stride-n access to B is thrashing the cache. Fix: reorder the loops to i-k-j for stride-1 access. Measure again: cache misses drop tenfold, IPC triples, a 10x speedup. Round two: perf now shows the program is closer to compute-bound. Hypothesis: temporal locality can still improve. Fix: tile the loops to keep working sets in L1. Measure again: a further modest gain. Round three: perf on the OpenMP version shows scheduling overhead. Fix: adjust the schedule clause. Measure again. Each round is one hypothesis, one change, one measurement, and the bottleneck migrates each time, exactly as we saw the GPU stalls migrate from memory to synchronization in Chapter 14. You stop when you hit the roofline or when the remaining gain is not worth the effort.

The same loop runs on the project. Dorado round one: Nsight shows the attention kernel is memory-bandwidth-bound. Hypothesis: the attention is re-reading data from global memory that could be reused. Fix: the Signal-to-Base cache to reduce redundant traffic. Measure again: DRAM throughput drops, the kernel climbs the roofline. Kraken-2 round one: Cachegrind shows a 78 percent LLC miss rate on the hash lookup. Hypothesis: the hot k-mers are scattered across the 180 GB table with no locality. Fix: the Hot-K-mer LRU cache to keep frequent k-mers local. Measure again: the miss rate on the hot path drops, the lookup speeds up. The loop is the same; only the tools and the workload change. That sameness is the deepest lesson of the book: profiling is one discipline, applied with different instruments to different machines, always measuring, always hypothesizing, always confirming before believing.

**How This Connects to Your Project**

The decision tree, the discipline, and the loop are the operating manual for the whole Nanopore profiling effort. Triage Dorado and Kraken-2 with the cheap tools, follow the tree to the right deep tool for each (Nsight for Dorado, perf and Cachegrind for Kraken-2), run the optimization loop on each (Signal-to-Base cache for Dorado, Hot-K-mer cache for Kraken-2), ground every number as a fraction of measured peak, and write it up with the Chapter 19 structure. Twenty chapters reduce to this practice, and this practice produces the two-page report. That is the arc of the book, and you have now walked all of it.

## 20.4 Appendix A: The Benchmark Suite

The five benchmark programs used throughout the book, for reference. The four CPU programs and the CUDA program, each the vehicle for the chapters that profiled them.

### matmul_naive.c (the worst-case baseline)

```c
#define N 1024
static void matmul_naive(const double *A, const double *B,
                         double *C, int n) {
    for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
        for (int k = 0; k < n; k++)
          C[i*n + j] += A[i*n + k] * B[k*n + j];
          // B[k*n+j]: stride-n access, a cache miss every inner iteration
}
```

### matmul_ikj.c (loop reorder fixes B access)

```c
static void matmul_ikj(const double *A, const double *B,
                       double *C, int n) {
    for (int i = 0; i < n; i++)
      for (int k = 0; k < n; k++) {
        double a_ik = A[i*n + k];          // hoist: load once, reuse n times
        for (int j = 0; j < n; j++)
          C[i*n + j] += a_ik * B[k*n + j]; // B now stride-1: prefetcher happy
      }
}
```

### matmul_tiled.c (blocking for L1)

```c
#define TILE_SIZE 32
static void matmul_tiled(const double *A, const double *B,
                         double *C, int n) {
    for (int ii = 0; ii < n; ii += TILE_SIZE)
    for (int kk = 0; kk < n; kk += TILE_SIZE)
    for (int jj = 0; jj < n; jj += TILE_SIZE)
      for (int i = ii; i < ii+TILE_SIZE && i < n; i++)
      for (int k = kk; k < kk+TILE_SIZE && k < n; k++) {
        double a_ik = A[i*n + k];
        for (int j = jj; j < jj+TILE_SIZE && j < n; j++)
          C[i*n + j] += a_ik * B[k*n + j];
      }
}
```

## matmul_omp.c (OpenMP over rows)

```c
static void matmul_omp(const double *A, const double *B,
                       double *C, int n) {
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < n; i++)              // rows split across threads
      for (int k = 0; k < n; k++) {          //  -> disjoint writes, no race
        double a_ik = A[i*n + k];
        for (int j = 0; j < n; j++)
          C[i*n + j] += a_ik * B[k*n + j];
      }
}
// compile: gcc -O2 -fopenmp matmul_omp.c -o matmul_omp
// run:     OMP_NUM_THREADS=8 ./matmul_omp
```

## matmul_cuda.cu (naive and tiled GPU kernels)

CUDA

```
#define N 1024
#define TILE 16
// naive: every thread reads A and B from global memory, no reuse
__global__ void matmul_naive_gpu(const float *A, const float *B,
                                 float *C, int n) {
    int row = blockIdx.y*blockDim.y + threadIdx.y;
    int col = blockIdx.x*blockDim.x + threadIdx.x;
    if (row >= n || col >= n) return;
    float sum = 0.0f;
    for (int k = 0; k < n; k++)
        sum += A[row*n + k] * B[k*n + col];  // warp: consecutive col => coalesced
    C[row*n + col] = sum;
}
// tiled: cooperative shared-memory staging, GMEM traffic / TILE
__global__ void matmul_tiled_gpu(const float *A, const float *B,
                                 float *C, int n) {
    __shared__ float sA[TILE][TILE], sB[TILE][TILE];
    int row = blockIdx.y*TILE + threadIdx.y;
    int col = blockIdx.x*TILE + threadIdx.x;
    float sum = 0.0f;
    for (int ph = 0; ph < (n+TILE-1)/TILE; ph++) {
        sA[threadIdx.y][threadIdx.x] =
          (row<n && ph*TILE+threadIdx.x<n) ? A[row*n+ph*TILE+threadIdx.x] : 0;
        sB[threadIdx.y][threadIdx.x] =
          (ph*TILE+threadIdx.y<n && col<n) ? B[(ph*TILE+threadIdx.y)*n+col] : 0;
        __syncthreads();                        // all loaded before compute
        for (int t = 0; t < TILE; t++)
            sum += sA[threadIdx.y][t] * sB[t][threadIdx.x];
        __syncthreads();                        // all done before overwrite
    }
    if (row<n && col<n) C[row*n + col] = sum;
}
// compile: nvcc -O3 -lineinfo -o matmul_cuda matmul_cuda.cu
```

## 20.5 Appendix B: Command Reference Sheet

The commands from across the book, in one place, as a working reference.

```bash
# --- perf (Batch 1) ---
perf stat ./prog                       # hardware counters, IPC, miss rate
perf stat -r 5 ./prog                  # 5 runs with stddev
perf stat -e cycles,instructions,LLC-loads,LLC-load-misses ./prog
perf record -g ./prog ; perf report    # sampling hotspot profile
perf annotate <function>               # source+assembly per-line samples
perf record -e cache-misses -g ./prog  # sample weighted by cache misses
perf record -F 99 -g ./prog            # 99 Hz for flame graphs
perf diff before.data after.data       # before/after comparison
perf mem record ./prog ; perf mem report   # per-access memory level
perf c2c record ./prog ; perf c2c report   # false sharing
perf bench mem memcpy                  # measure memcpy bandwidth

# --- Valgrind (Batch 2) ---
valgrind --tool=cachegrind --cachegrind-out-file=cg.out ./prog
cg_annotate cg.out                     # per-line cache counts
valgrind --tool=callgrind --instr-atstart=no ./prog   # profile hot region
callgrind_control -i on ; callgrind_control -d
valgrind --leak-check=full --track-origins=yes ./prog # memcheck
valgrind --tool=massif --pages-as-heap=yes ./prog ; ms_print massif.out.PID
valgrind --tool=helgrind ./prog        # data race detection

# --- gprof (Batch 2) ---
gcc -O2 -pg -g -o prog prog.c ; ./prog ; gprof prog gmon.out

# --- Nsight (Batch 3) ---
nsys profile --trace=cuda,nvtx,osrt --output=prof ./prog
nsys stats --report cuda_gpu_kern_sum prof.nsys-rep
ncu --kernel-name <name> --launch-count 1 --set full ./prog

# --- VTune / LIKWID (Batch 4) ---
vtune -collect uarch-exploration -result-dir r ./prog   # Top-Down
likwid-perfctr -C 0 -g CACHE ./prog    # curated counter group
likwid-bench -t stream -w S0:512MB     # measured peak bandwidth

# --- system tools (Batch 4) ---
/usr/bin/time -v ./prog                # triage: cpu/mem/io/faults
vmstat 1 ; iostat -x 1                  # swap and I/O pressure
strace -c ./prog                       # syscall summary
numactl --hardware                     # NUMA topology
```

## 20.6 Appendix C: Common perf Output Patterns

The recurring perf stat patterns and what each means, as a diagnostic quick-reference. When you see the pattern on the left, the diagnosis on the right is your starting hypothesis.

output

```
Pattern (perf stat)                      Diagnosis
-------------------------------------    --------------------------
IPC < 0.5, LLC miss rate > 30%           memory-bandwidth-bound;
                                         reduce traffic (tile, cache)
IPC < 0.5, LLC miss rate low,            pointer-chasing / latency-
  high backend stalls                    bound; improve locality
IPC > 2.5, low miss rate                 compute-bound; near ceiling,
                                         optimize algorithm not memory
branch-miss rate > 5%                    unpredictable branches;
                                         data-dependent control flow
high page-faults, major faults > 0       swapping or first-touch;
                                         check vmstat, add memory
high context-switches/migrations         OS interference or over-
                                         subscription; pin threads
task-clock >> wall time (multithreaded)  good: threads using many
                                         cores in parallel
task-clock ~= wall time (multithreaded)  bad: threads serialized;
                                         check locks (perf c2c, vtune)
```

Keep this table close in your early profiling work. With time the patterns become reflexive: you glance at a perf stat output and the diagnosis arrives before you have consciously read the numbers, the way an experienced doctor reads an EKG. That reflex is the goal of the whole book. The tools are means; the end is the trained judgment that looks at a slow program, reaches for the right instrument, reads the result against the right ceiling, and knows what to do. You now have the means. The judgment comes from running the loop, on matmul, on Kraken-2, on Dorado, on everything you profile from here. Go measure.

### What We Just Learned

The tool-selection decision tree collapses the whole book into one table: cheap system tools to triage, perf and Valgrind for the CPU core, VTune and LIKWID as specialized CPU lenses, Nsight for the GPU (survey then microscope), gprof as the constrained fallback. Memorize its shape.

The profiling discipline: never profile a debug build, never report a single run, never ignore tool overhead, never optimize what the profiler does not confirm is hot. Always triage cheap-to-expensive, express performance as a fraction of measured peak, compile with -g or -lineinfo, and form a hypothesis before measuring.

Profiling is a loop: measure, hypothesize, fix, measure again, repeat until the roofline or out of time. The bottleneck migrates each round, as it did from memory to synchronization on the GPU and from cache to compute on the CPU.

The same loop runs on the project: Signal-to-Base cache for Dorado's memory-bound attention, Hot-K-mer cache for Kraken-2's latency-bound lookup. One discipline, different instruments, different machines.

The appendices collect the benchmark source, the command reference, and the perf output patterns, for use long after the chapters fade.

The end of the book is not knowing the tools but the trained judgment that wields them: look at a slow program, reach for the right instrument, read against the right ceiling, know what to do. The means are now yours; the judgment comes from running the loop.

### A Final Word

You started this book with a friend who spent three weeks optimizing the wrong function because he guessed instead of measuring. You end it able to never make that mistake: to triage in seconds, localize in minutes, diagnose against a measured ceiling, and recommend the fix the evidence demands, on either side of the PCIe bus.

The two-page report for Prof. Paul is no longer a daunting deliverable; it is the natural output of the practice you now hold. Profile Dorado, profile Kraken-2, run the loop, ground every number, write it up. The tools were never the point. The judgment was. Go measure, and trust the numbers over the guess, every time.